

Spring WebFlux Complete Guide

Table of Contents

1. [Reactive Programming Fundamentals](#reactive-fundamentals)
2. [Spring WebFlux Core Concepts](#webflux-core)
3. [Project Reactor Essentials](#project-reactor)
4. [WebFlux APIs and Routing](#webflux-apis)
5. [Testing Reactive Applications](#testing)
6. [Debugging and Troubleshooting](#debugging)
7. [Best Practices](#best-practices)

Reactive Programming Fundamentals

Understanding Reactive Streams

Reactive Streams core interfaces:

```
```java
```

```
Public interface Publisher<T> {
 Void subscribe(Subscriber<? Super T> subscriber);
}
```

```
Public interface Subscriber<T> {
 Void onSubscribe(Subscription s);
 Void onNext(T t);
 Void onError(Throwable t);
 Void onComplete();
}
```

```
Public interface Subscription {
```

```
 Void request(long n);
```

```
 Void cancel();
```

```
}
```

```
...
```

```
Backpressure Handling
```

```
```java
```

```
Flux.range(1, 1000000)
```

```
    .onBackpressureBuffer(10000) // Buffer size
```

```
    .subscribe(new BaseSubscriber<Integer>() {
```

```
        @Override
```

```
        Protected void hookOnSubscribe(Subscription subscription) {
```

```
            Request(100); // Request initial batch
```

```
        }
```

```
        @Override
```

```
        Protected void hookOnNext(Integer value) {
```

```
            processValue(value);
```

```
            request(1); // Request one more
```

```
        }
```

```
    });
```

```
...
```

```
## Project Reactor Core
```

```
### Flux and Mono Basics
```

Creating Flux:

```
```java
// Various ways to create Flux
Flux<String> flux1 = Flux.just("A", "B", "C");
Flux<Integer> flux2 = Flux.fromArray(new Integer[]{1, 2, 3});
Flux<Long> flux3 = Flux.interval(Duration.ofMillis(100));

// Flux operations
Flux1.map(String::toLowerCase)
 .filter(s -> s.length() > 0)
 .distinct()
 .subscribe(System.out::println);
```
```

Creating Mono:

```
```java
Mono<String> mono1 = Mono.just("Single Value");
Mono<String> mono2 = Mono.empty();
Mono<String> mono3 = Mono.error(new RuntimeException("Error"));

// Mono operations
Mono1.map(String::toUpperCase)
 .flatMap(this::processValue)
 .subscribe(
 Value -> System.out.println("Value: " + value),
 Error -> System.err.println("Error: " + error),
 () -> System.out.println("Completed")
);
```
```

```

### ### Operators

#### #### Transformation Operators

```java

```
Flux<String> names = Flux.just("John", "Jane", "Bob");
```

// Map transformation

```
Flux<Integer> nameLengths = names.map(String::length);
```

// FlatMap for async operations

```
Flux<UserData> userDataFlux = names.flatMap(name ->
```

```
    webClient.get()
```

```
        .uri("/users/" + name)
```

```
        .retrieve()
```

```
        .bodyToMono(UserData.class)
```

```
);
```

// Zip operation

```
Flux.zip(nameLengths, userDataFlux)
```

```
    .map(tuple -> new CombinedData(tuple.getT1(), tuple.getT2()));
```

```

#### #### Filtering Operators

```java

```
Flux<Integer> numbers = Flux.range(1, 100);
```

```
Numbers.filter(n -> n % 2 == 0)    // Even numbers
```

```
    .take(10)                      // First 10 elements
```

```
    .takeUntil(n -> n > 50)       // Take until condition
```

```
    .distinct()                   // Remove duplicates
```

```
    .subscribe(System.out::println);
```

```
    ````
```

#### #### Error Handling

```
````java
```

```
Flux<String> data = processData()
```

```
    .onErrorResume(error -> {
```

```
        Log.error("Error occurred: ", error);
```

```
        Return Flux.empty();
```

```
    })
```

```
    .onErrorMap(error ->
```

```
        New CustomException("Processing failed", error)
```

```
    )
```

```
    .doOnError(error ->
```

```
        Metrics.incrementErrorCount()
```

```
    );
```

```
    ````
```

#### ## Spring WebFlux Implementation

##### ### Router Functions

```
````java
```

```
@Configuration
```

```
Public class RouterConfig {
```

@Bean

```
Public RouterFunction<ServerResponse> route(UserHandler userHandler) {
```

```
    Return RouterFunctions
```

```
        .route(GET("/users").and(accept(APPLICATION_JSON)),
```

```
            userHandler::getAllUsers)
```

```
        .andRoute(GET("/users/{id}").and(accept(APPLICATION_JSON)),
```

```
            userHandler::getUser)
```

```
        .andRoute(POST("/users").and(contentType(APPLICATION_JSON)),
```

```
            userHandler::createUser)
```

```
        .andRoute(PUT("/users/{id}").and(contentType(APPLICATION_JSON)),
```

```
            userHandler::updateUser)
```

```
        .andRoute(DELETE("/users/{id}"),
```

```
            userHandler::deleteUser);
```

```
    }
```

```
}
```

```
...
```

Handler Implementation

```
```java
```

@Component

```
Public class UserHandler {
```

```
 Private final UserService userService;
```

```
 Public Mono<ServerResponse> getAllUsers(ServerRequest request) {
```

```
 Return ServerResponse.ok()
```

```
 .contentType(APPLICATION_JSON)
```

```
 .body(userService.getAllUsers(), User.class);
```

```
 }
```

```

Public Mono<ServerResponse> getUser(ServerRequest request) {
 String userId = request.pathVariable("id");
 Return userService.getUser(userId)
 .flatMap(user ->
 ServerResponse.ok()
 .contentType(APPLICATION_JSON)
 .bodyValue(user)
)
 .switchIfEmpty(ServerResponse.notFound().build());
}

```

```

Public Mono<ServerResponse> createUser(ServerRequest request) {
 Return request.bodyToMono(User.class)
 .flatMap(user ->
 userService.createUser(user)
 .flatMap(savedUser ->
 ServerResponse.created(URI.create("/users/" + savedUser.getId()))
 .contentType(APPLICATION_JSON)
 .bodyValue(savedUser)
)
);
}
}
` ``

```

### Service Layer

`` `java

@Service

Public class UserService {

Private final UserRepository userRepository;

Public Flux<User> getAllUsers() {

Return userRepository.findAll()

.timeout(Duration.ofSeconds(5))

.onErrorResume(TimeoutException.class,

Ex -> Flux.error(new ServiceException("Database timeout")));

}

Public Mono<User> getUser(String id) {

Return userRepository.findById(id)

.switchIfEmpty(Mono.error(

New UserNotFoundException("User not found: " + id)

));

}

Public Mono<User> createUser(User user) {

Return validateUser(user)

.then(userRepository.save(user))

.doOnSuccess(savedUser ->

eventPublisher.publishUserCreated(savedUser));

}

}

...

### WebClient Usage



```
```java
```

```
@Configuration
```

```
Public class WebClientConfig {
```

```
    @Bean
```

```
    Public WebClient webClient() {
```

```
        Return WebClient.builder()
```

```
            .baseUrl(https://api.example.com)
```

```
            .defaultHeader(HttpHeaders.CONTENT_TYPE,  
                MediaType.APPLICATION_JSON_VALUE)
```

```
            .filter(logRequest())
```

```
            .filter(logResponse())
```

```
            .build();
```

```
    }
```

```
    Private ExchangeFilterFunction logRequest() {
```

```
        Return ExchangeFilterFunction.ofRequestProcessor(clientRequest -> {
```

```
            Log.debug("Request: {} {}",
```

```
                clientRequest.method(),
```

```
                clientRequest.url());
```

```
            return Mono.just(clientRequest);
```

```
        });
```

```
    }
```

```
}
```

```
@Service
```

```
Public class ExternalService {
```

```
    Private final WebClient webClient;
```

```

Public Mono<ExternalData> fetchExternalData(String id) {
    Return webClient.get()
        .uri("/external/" + id)
        .retrieve()
        .bodyToMono(ExternalData.class)
        .timeout(Duration.ofSeconds(5))
        .retryWhen(Retry.backoff(3, Duration.ofSeconds(1)))
        .onErrorResume(WebClientResponseException.NotFound.class,
            Ex -> Mono.empty());
}
}
` ``

```

Testing Reactive Applications

Unit Testing

```

` `` java

@ExtendWith(MockitoExtension.class)

Class UserServiceTest {

    @Mock

    Private UserRepository userRepository;

    @InjectMocks

    Private UserService userService;

    @Test

    Void testGetUser() {

        String userId = "123";
    }
}

```

```
User user = new User(userId, "Test User");
```

```
When(userRepository.findById(userId))
```

```
    .thenReturn(Mono.just(user));
```

```
StepVerifier.create(userService.getUser(userId))
```

```
    .expectNext(user)
```

```
    .verifyComplete();
```

```
}
```

```
@Test
```

```
Void testUserNotFound() {
```

```
    String userId = "nonexistent";
```

```
When(userRepository.findById(userId))
```

```
    .thenReturn(Mono.empty());
```

```
StepVerifier.create(userService.getUser(userId))
```

```
    .expectError(UserNotFoundException.class)
```

```
    .verify();
```

```
}
```

```
}
```

```
...
```

```
### Integration Testing
```

```
```java
```

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
```

```
Class UserControllerIntegrationTest {
```

@Autowired

Private WebTestClient webTestClient;

@Test

Void testCreateUser() {

    User user = new User(null, "New User");

    webTestClient.post()

        .uri("/users")

        .contentType(MediaType.APPLICATION\_JSON)

        .bodyValue(user)

        .exchange()

        .expectStatus().isCreated()

        .expectBody()

        .jsonPath("\$.name").isEqualTo("New User")

        .jsonPath("\$.id").isNotEmpty();

}

@Test

Void testGetAllUsers() {

    webTestClient.get()

        .uri("/users")

        .accept(MediaType.APPLICATION\_JSON)

        .exchange()

        .expectStatus().isOk()

        .expectHeader().contentType(MediaType.APPLICATION\_JSON)

        .expectBodyList(User.class)

        .hasSize(greaterThan(0));

```
}
}
```
```

Performance Testing

```
```java  
Class PerformanceTest {
 @Test
 Void loadTest() {
 Int numberOfRequests = 1000;
 Flux.range(1, numberOfRequests)
 .flatMap(i -> webClient.get()
 .uri("/users")
 .retrieve()
 .bodyToMono(String.class))
 .blockLast(Duration.ofMinutes(1));
 }
}
```
```

Debugging and Troubleshooting

Logging Configuration

```
```java  
Flux<String> data = sourceFlux
 .doOnSubscribe(s -> log.debug("Subscribed"))
 .doOnNext(value -> log.debug("Processing: {}", value))
 .doOnError(error -> log.error("Error: ", error))
```

```
.doOnComplete() -> log.debug("Completed"))
.log(); // Reactor's built-in logging
...

```

### ### Hooks for Debugging

```
```java
// Global debug mode
Hooks.onOperatorDebug();

// Custom hooks
Hooks.onNextDropped(value ->
    Log.warn("Dropped value: {}", value));
Hooks.onErrorDropped(error ->
    Log.error("Dropped error: {}", error));

// Thread name for debugging
Flux.just("value")
    .publishOn(Schedulers.boundedElastic())
    .doOnNext(value ->
        Log.debug("Thread: {}", Thread.currentThread().getName()));
...

```

Common Issues and Solutions

Memory Leaks

```
```java
// Prevent memory leaks with proper subscription handling
Public class ResourceManager {

```

```
Private final List<Disposable> subscriptions = new ArrayList<>();
```

```
Public void startProcess() {
```

```
 Disposable subscription = Flux.interval(Duration.ofSeconds(1))
```

```
 .subscribe();
```

```
 Subscriptions.add(subscription);
```

```
}
```

```
Public void cleanup() {
```

```
 Subscriptions.forEach(Disposable::dispose);
```

```
 Subscriptions.clear();
```

```
}
```

```
}
```

```
...
```

#### #### Timeout Handling

```
```java
```

```
Public Mono<Data> fetchDataWithTimeout() {
```

```
    Return webClient.get()
```

```
        .uri("/data")
```

```
        .retrieve()
```

```
        .bodyToMono(Data.class)
```

```
        .timeout(Duration.ofSeconds(5))
```

```
        .onErrorResume(TimeoutException.class, ex -> {
```

```
            Log.warn("Request timed out");
```

```
            Return Mono.empty();
```

```
        });
```

```
}
```

```

## ## Best Practices

### ### Resource Management

```java

@Component

Public class DatabaseClient {

Private final ConnectionPool pool;

Public Mono<Connection> getConnection() {

Return Mono.fromSupplier(() -> pool.acquire())

.doOnNext(conn -> log.debug("Acquired connection"))

.doOnError(e -> log.error("Failed to acquire connection", e))

.doFinally(signal -> {

If (signal == Signal.ON_ERROR || signal == Signal.ON_COMPLETE) {

Pool.release();

}

});

}

}

```

### ### Error Handling Patterns

```java

Public class ErrorHandlingPatterns {

// Circuit breaker pattern

Public Mono<Data> fetchWithCircuitBreaker() {


```

Return CircuitBreaker.create("fetch-data")

    .run(webClient.get()

        .uri("/data")

        .retrieve()

        .bodyToMono(Data.class),

        Throwable -> fallbackMethod());
}

```

// Retry pattern

```

Public Mono<Data> fetchWithRetry() {

    Return webClient.get()

        .uri("/data")

        .retrieve()

        .bodyToMono(Data.class)

        .retryWhen(Retry.backoff(3, Duration.ofSeconds(1))

            .maxBackoff(Duration.ofSeconds(5))

            .doBeforeRetry(signal ->

                Log.warn("Retrying after error: {}",

                    Signal.failure().getMessage())));

}

}

...

```

Performance Optimization

```java

@Configuration

Public class WebFluxConfig {

@Bean

```

Public ReactorResourceFactory reactorResourceFactory() {
 ReactorResourceFactory factory = new ReactorResourceFactory();
 Factory.setUseGlobalResources(false);
 Factory.setConnectionProvider(
 ConnectionProvider.builder("http")
 .maxConnections(500)
 .pendingAcquireTimeout(Duration.ofSeconds(60))
 .build()
);
 Return factory;
}

```

@Bean

```

Public HttpClient httpClient(ReactorResourceFactory resourceFactory) {
 Return HttpClient.create(resourceFactory.getConnectionProvider())
 .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, 10000)
 .responseTimeout(Duration.ofSeconds(10))
 .doOnConnected(conn -> conn
 .addHandlerLast(new ReadTimeoutHandler(10))
 .addHandlerLast(new WriteTimeoutHandler(10)));
 }
}
...

```

## Conclusion

This guide covers the essential aspects of Spring WebFlux and reactive programming.  
Key takeaways:

1. Understand reactive streams and backpressure
2. Master Project Reactor operators
3. Implement proper error handling
4. Write comprehensive tests
5. Monitor and debug effectively
6. Follow best practices for performance

Keep practicing and experimenting with these concepts to build robust reactive applications.

Remember:

- Start with simple flows and gradually add complexity
- Always handle errors appropriately
- Test thoroughly, especially edge cases
- Monitor performance in production
- Keep learning and staying updated with new features